Lecture 8: Sequential Logic Circuits

Lui Holder-Pearson

September 17, 2026

Learning Goals

- Understand the design of:
  - Shift registers;
  - Counters; and
  - Finite state machines.
- Understand the role of the above within embedded systems.

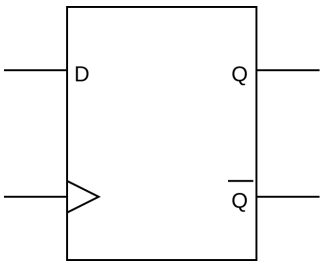
Further Reading

Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapter 14.2
Youtube videos visualising:
Shift registers
Counters

Recap: Sequential logic

Way back in week 1, Lecture 3 - Memory & buses, we discussed sequential logic. Sequential logic differentiates from combinational logic in that the output is a function of the current state of the system, not only the inputs. These functions therefore require determination of the output for all combinations of inputs and current states.

One of the fundamental building blocks for sophisticated sequential logic circuits are the flip-flop. The flip-flop is similar to an SR-latch, with additional circuitry for edge-detection.

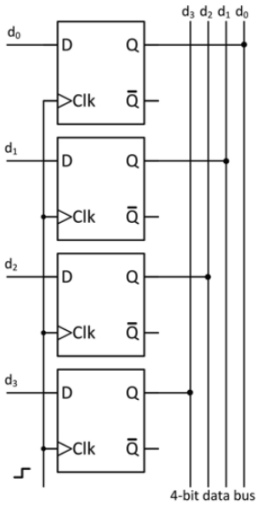


Where `D` is input data that is captured on a **rising edge** of an input signal, typically a clock.

A clock, or `CLK`, is a square wave with a very precise frequency. Clocks form the basis of all timing in computer systems.

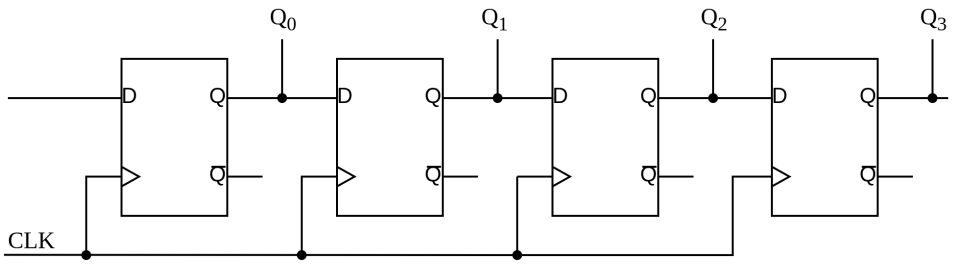
1 More complicated sequential circuits

Recall the register, which is a one-dimensional array of flip-flops:



The first circuit to look at is the shift register:

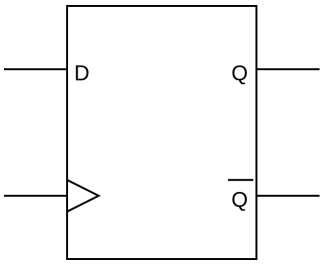
- The output of each flip-flop is connected to the input of the adjacent flip-flop.
- All of the flip-flops share a common `CLK`.
- This is one hardware method of implementing a first in, first out (FIFO) buffer.
- Another use is for parallel-to-serial peripherals, for example to allow communication to be sent.



The shift register is useful for converting between serial and parallel communication systems. The `CLK` source can be from an external device, and therefore communication can occur between systems with different frequency capabilities.

1.1 Counters

There are specific cases in which we use the `CLK` input of a flip-flop for other uses. Let us consider a slightly unintuitive use of a flip flop:



By connecting the input — `D` — to the inverted output — $\overline{\texttt{Q}}$ — of the flip-flop, we had made something that halves the clock rate: a *prescaler*.

If we chain several together such that the output of one becomes the `CLK` of the next, we get a familiar pattern appearing (simulated here):

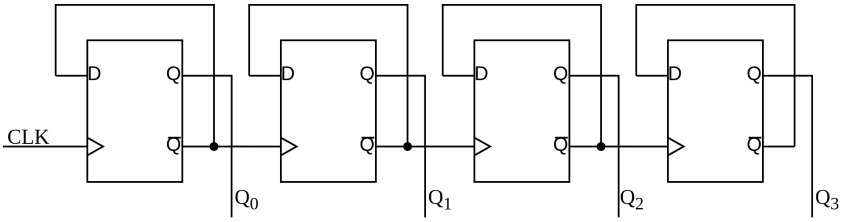


Figure 1: A 4-bit ripple counter

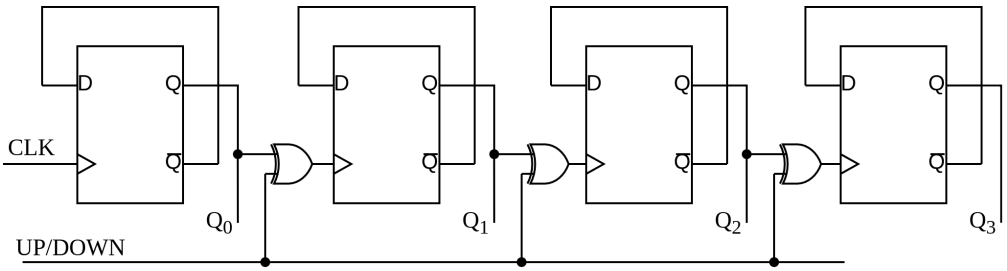
.



State	Q_3	Q_2	Q_1	Q_0
0				
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				

Counters are extremely useful, for example for allowing ALUs to perform multiplication and division, scaling clocks down, or for implementing different timers within an embedded system processor.

Down counters can be implementing by connecting `Q` to the adjacent `CLK`, instead of $\overline{\texttt{Q}}$. An electrically-configurable counter is able to be implemented with an options inversion, using the same inversion technique as subtraction:



1.2 Real world propagation delays

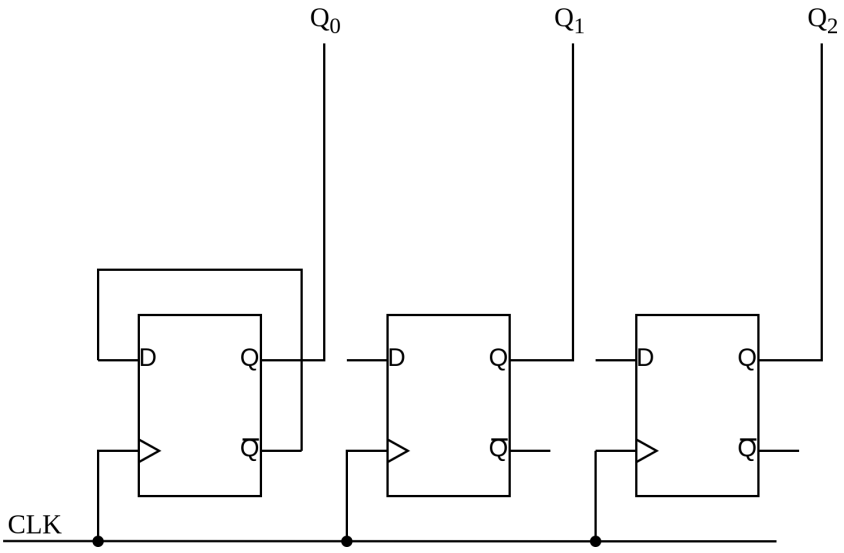
Unfortunately, we must take the real world into consideration. Specifically, transistor propagation delay. Propagation delay is the time it takes for transistors to switch off or on. It is a very small, but non-zero amount of time. For example, modern transistors within CPUs have a propagation delay of just 200 ps, or 0.0000000002 s.

1.2.1 Synchronous counter

Sequential logic circuits are synchronous if they will always change state at the same time, i.e. if:

Asynchronous sequential logic circuits are disadvantageous because the multiple very small propagation delays add together Through the use of some external circuitry, this can be rectified. We will design this circuitry using a truth table:

$Q_{2,i}$	$Q_{1,i}$	$Q_{0,i}$	$Q_{2,i+1}$	$Q_{1,i+1}$	$Q_{0,i+1}$
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

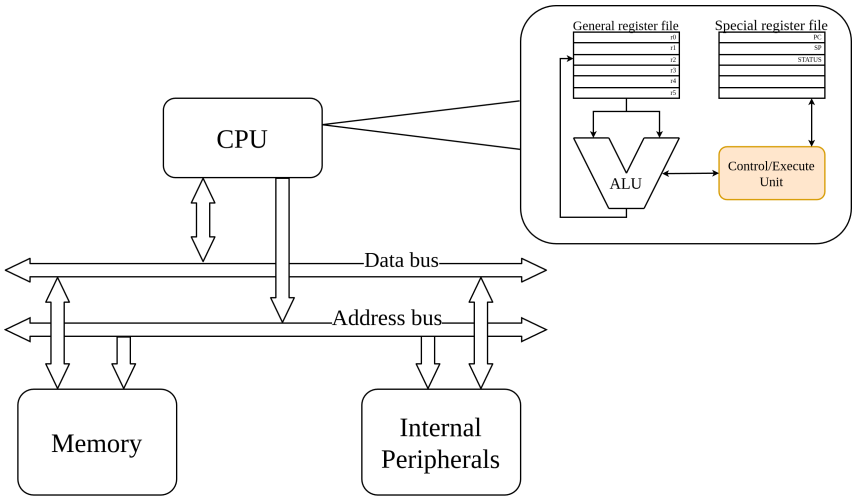


1.3 Finite state machines

Finite state machines (FSM) are a tool used to encode control systems that exist in specific, limited states. Common, every-day examples include:

- Air conditioning unit
- Car gearboxes
- Elevator controllers
- Torches
- Watches

Within a computer, specific states can be encoded in series of flip-flops. Counters are an example of state machines where the state increments every rising clock edge. One specific use of FSMs inside an embedded system is in the implementation of the control/execute unit of the CPU.



A watch is an example of a FSM. Looking solely at the stopwatch functionality of my Casio F91-W, the states are:

Simplified:



1.
2.
3.

Because there are a finite number of states, it is possible to directly represent them with diagrams. The important aspects are:

- Initial state
- Inputs
- Transitions
- Outputs

A finite state machine is defined by:

- Number of outputs:
  - If only one output is true at any time, it is known as a one-hot FSM.
  - Otherwise, the output state can be encoded in several outputs.
- Output reliance on:
  - *state variables* only, or
  - *state variables* as well as inputs.

Discrete component representation of FSMs:

Exercises:

1. How would you implement a circuit to detect the falling edge of a clock?
2. Draw the finite state machine for the entire functionality of my Casio F91-W.

Summary

- Understand the design of:
  - Shift registers;
    → Shift registers are when the output of one flip-flop is connected to the input of the next, with a common clock. Used for serial-parallel conversions.
  - Counters; and
    → Counters use the inverted output of the flip-flop as the input. This therefore acts as a clock divider, or prescaler. Multiple connected flip-flops form a ripple counter.
    → Synchronous counters use additional circuitry to predict the 'next' state of the system as a function of the current state.
  - Finite state machines.
    → Finite state machines are control systems used to specify the state of a system.
- Understand the role of the above within embedded systems.
  → Each of them are contained in some format within a microcontroller to achieve base, or additional functionality.

Next Lecture: Lecture 9 - Peripherals

2 Questions?